



Factory Method

Instituto Federal do Mato Grosso (IFMT)
Pontes e Lacerda - Fronteira Oeste



Conteúdo da aula

- **Introdução**
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Introdução





Conteúdo da aula

- **Introdução**
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Revisão de Python



- **Classe Abstrata**
 - **Definição:** É uma classe não instanciável, usada para agrupar um conjunto de características comuns entre subclasses.
 - Ela é usada como base para criação de subclasses concretas (instanciável)
 - Fornecer um comportamento base e consistente entre as subclasses.

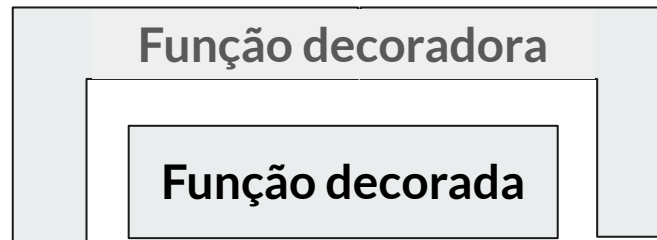
Revisão de Python



- **Classe Abstrata**
 - ***Vantagens:***
 - Reutilização de código
 - Comportamento unitário entre subclasses
 - Organização do código
 - Em python é necessário importar dois elementos
 - A classe **ABC** (Abstract Basic Class)
 - E o decorator **abstractmethod**

Revisão de Python

- **Classe ABC**: Garante que a própria classe abstrata, e qualquer subclasse que não implementa todos os métodos abstratos, não poderá ser instanciada.
- **Decorator**: Uma função que serve para aprimorar outra função
 - A função decoradora recebe a outra função como argumento



Revisão de Python



```
# A simple decorator function
def decorator(func):

    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper

# Applying the decorator to a function
@decorator

def greet():
    print("Hello, World!")

greet()
```

Revisão de Python



```
Before calling the function.  
Hello, World!  
After calling the function.
```

Fonte: <https://www.geeksforgeeks.org/python/decorators-in-python/>

- Decorator **@abstractmethod** determina que as subclasses devem implementar o método a qual ele está acoplado

Revisão de Python

```
1  from abc import ABC, abstractmethod
2
3  class ExemploClasseAbstrata (ABC):
4
5      @abstractmethod
6      def imprimeValor(self):
7          pass
8
9      def imprimeAlgo(self, mensagem):
10         print(mensagem)
```

Revisão de Python

```
12 class ExemploClasseConcreta (ExemploClasseAbstrata):
13
14     def __init__(self, valor):
15         self.valor = valor
16
17     def imprimeValor(self):
18         print(self.valor)
19
20 c = ExemploClasseConcreta(15)
21 c.imprimeAlgo("Valor a seguir")
22 c.imprimeValor()
```



Conteúdo da aula

- **Introdução**
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

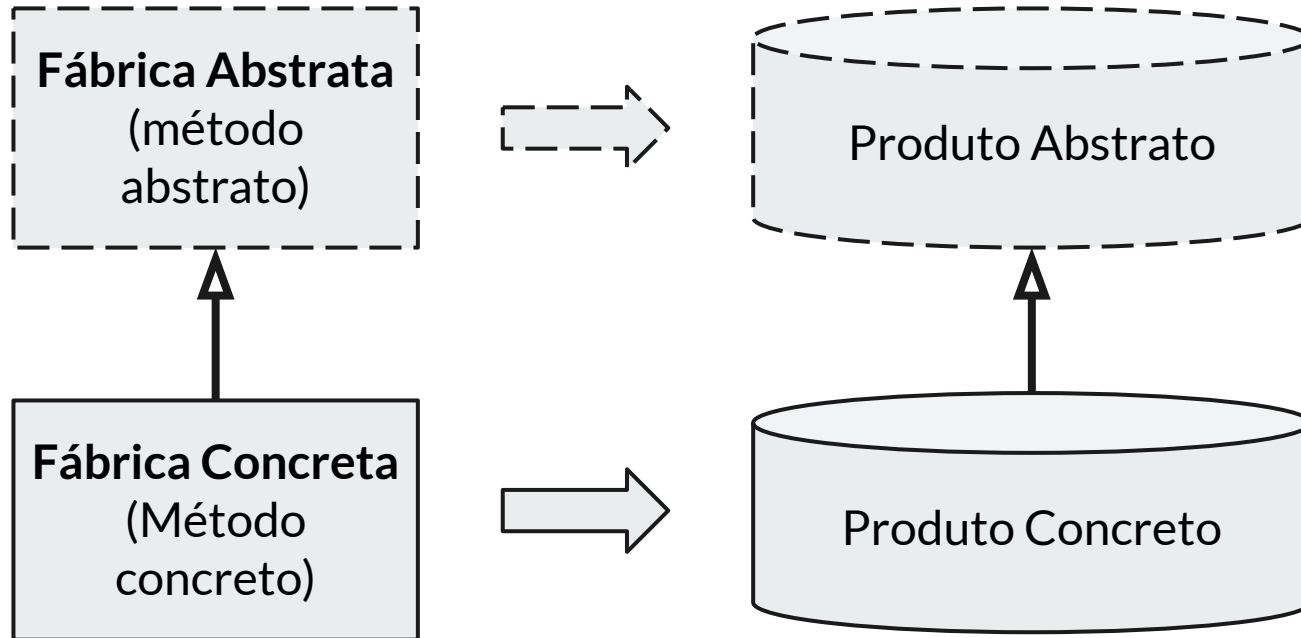
Factory Method



- **Intenção**: Criar uma interface (ou classe abstrata) que serve para criar outras classes. Porém, deixa com as subclasses a escolha de implementação da classe
- Quatro estruturas principais:
 - Fábrica abstrata → Cria um produto abstrato
 - Produto abstrato → Produto genérico usado na criação de outros produtos
 - Fábrica concreta → Fábrica que implementa o método de criação dos produtos
 - Produto concreto → Produto específico para determinado contexto

Factory Method

- Esquema visual genérico de um padrão factory method



Factory Method



- **Finalidades do padrão factory method:**
 - 1) Cria uma interface que disponibiliza o método de criação, no qual o cliente pode apenas usar o objeto, sem precisar saber qual subclasse gerou ele
 - 2) Quando é necessário gerar um objeto sem saber da classe concreta que gerou ele
 - 3) Quando faz sentido deixar que a subclasse decida qual implementação usar

Factory Method



```
from abc import ABC, abstractmethod

class Animal (ABC):

    def __init__(self, tamanho):
        self.tamanho = tamanho

    @abstractmethod
    def falar():
        pass

    def dormir(self):
        print("ZzzZzzZzzZzz...")
```

Factory Method



```
from Animal import Animal

class Cachorro(Animal):

    def falar(self):
        print("Au Au ...")
```

```
from Animal import Animal

class Gato (Animal):

    def falar(self):
        print("Miau Miau ...")
```

Factory Method

```
from abc import ABC, abstractmethod

class FabricaAbstrata(ABC):

    @abstractmethod
    def gerarAnimal(self, tamanho):
        pass
```

Factory Method

```
from FabricaAbstrata import FabricaAbstrata
from Cachorro import Cachorro

class FabricaCachorro (FabricaAbstrata):

    def gerarAnimal(self, tamanho):
        return Cachorro(tamanho)
```

Factory Method

```
from FabricaAbstrata import FabricaAbstrata
from Gato import Gato

class FabricaGato (FabricaAbstrata):

    def gerarAnimal(self, tamanho):
        return Gato(tamanho)
```

Factory Method

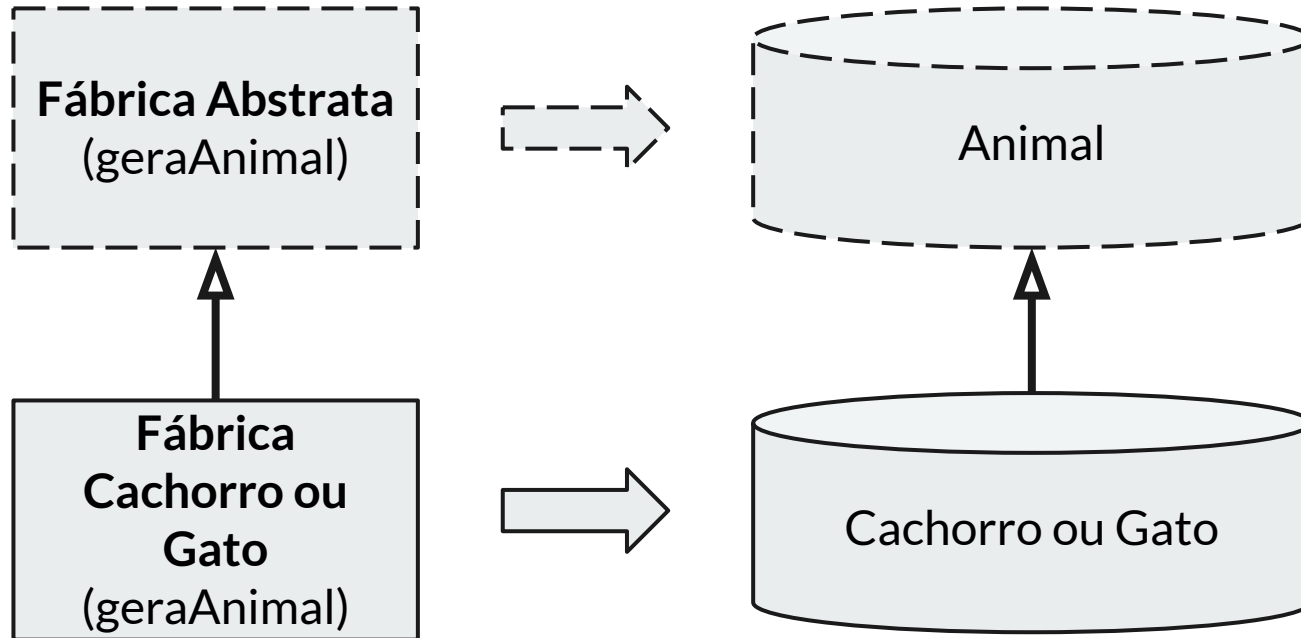
```
from Cachorro import Cachorro
from Gato import Gato
from FabricaCachorro import FabricaCachorro
from FabricaGato import FabricaGato
```

```
fabrica = FabricaCachorro()
animal = fabrica.gerarAnimal(1.0)
animal.falar()
animal.dormir()
```

```
fabrica = FabricaGato()
animal = fabrica.gerarAnimal(1.0)
animal.falar()
animal.dormir()
```

Factory Method

- Esquema visual genérico de um padrão factory method



Factory Method

```
package exemploFactoryMethod;  
  
public abstract class Pessoa {  
    public abstract void falar();  
}
```

Factory Method

```
package exemploFactoryMethod;

public class Chines extends Pessoa{

    @Override
    public void falar() {
        System.out.printf ("Mandarim%n");
    }
}
```

Factory Method

```
package exemploFactoryMethod;

public class Chileno extends Pessoa{

    @Override
    public void falar() {
        System.out.printf("Espanhol%n");
    }
}
```

Factory Method

```
package exemploFactoryMethod;  
  
public abstract class Creator {  
    public abstract Pessoa geraPessoa();  
}
```

Factory Method

```
package exemploFactoryMethod;

public class CreatorChileno extends Creator{

    @Override
    public Pessoa geraPessoa() {
        return new Chileno();
    }
}
```

Factory Method

```
package exemploFactoryMethod;

public class CreatorChines extends Creator{

    @Override
    public Pessoa geraPessoa() {
        return new Chines();
    }
}
```

Factory Method

```
public class PrincipalFactory {  
  
    public static void main(String[] args) {  
  
        Creator fabrica;  
  
        fabrica = new CreatorChines();  
        Pessoa chines = fabrica.geraPessoa();  
        chines.falar();  
  
        fabrica = new CreatorChileno();  
        Pessoa chileno = fabrica.geraPessoa();  
        chileno.falar();  
  
    }  
}
```



Conteúdo da aula

- Introdução
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Conclusão



Exercícios Propostos



1) Crie um sistema que use o Factory Method. O contexto é escolha do estudante

Bibliografia Básica



- GAMMA, Erich; et al. **Padrões de projeto: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2008.
- FREEMAN, E.; FREEMAN, E. **Use a Cabeça! Padrões de Projetos (Design Patterns)**. 2a ed. Alta Books, 2007.
- METSKER, S. J. **Padrões de projeto em Java**. Porto Alegre: Bookman, 2004.

Bibliografia Complementar



- NEIL, T. **Padrões de design para aplicativos móveis**. São Paulo: Novatec, 2012.
- SHALLOWAY, A.; TROTT, J. R. **Explicando padrões de projeto – uma nova perspectiva em projeto orientado a objetos**. Porto Alegre: Bookman, 2004.

Material de Apoio



- MENEZES, Nilo Ney Coutinho. **Introdução a Programação com Python: Algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.
- <https://www.datacamp.com/pt/tutorial/encapsulation-in-python-object-oriented-programming>
- https://medium.com/@nelson.miranda_40644/implementa%C3%A7%C3%A3o-do-pad%C3%A3o-de-design-singleton-em-python-um-guia-para-iniciantes-c033dcd773e5
- <https://www.datacamp.com/pt/tutorial/python-abstract-classes>
- <https://www.geeksforgeeks.org/python/decorators-in-python/>

Material de Apoio



- <https://medium.com/@ankitlodh2002/exploring-pythons-abstraction-abstract-methods-decorators-and-magic-methods-demystified-2c6ec5e14dce>
-